LAB

9

Operating Systems · FCIM / FAF · UTM · 2026-2027

# Watching System Calls with strace

**A** **Track A: the real OS, from the outside (Linux) · 90 minutes**

AT A GLANCE

| | | |
|---|---|---|
| **Part 1** | See the calls of a real command | ~20 min |
| **Part 2** | A program that calls the kernel directly | ~45 min |
| **Part 3** | Count one kind of call | ~20 min |

## Goal

In the lecture we crossed the door between a program and the kernel: the system call. Today you WATCH that door. strace prints every system call a program makes. You will read the calls of a real command, then write a tiny program that uses open, write and read directly, and watch its calls.

## Before you start

- Work on your own Linux laptop. Install strace if it is missing: sudo apt install strace (Debian/Ubuntu).
- Keep a file report.md open: every "Observe" box asks for one or two short answers there.

## PART 1 See the calls of a real command ~20 min

strace runs a command and prints each system call beside the normal output. Try it on a familiar command:

TERMINAL

```
$ strace -f -e trace=openat,read,write,close ls -l
```

You will see a burst of openat/read/close as ls reads the directory, then write as it prints. Count, roughly, how many system calls one simple ls makes.

**Observe (write in report.md)**

Which system call does ls use to print to the screen? To which descriptor does it write?

## PART 2 A program that calls the kernel directly ~45 min

Here is a short program that opens a file, writes a line, then reads it back, using the system calls open, write and read (no printf for the file itself). Type it, compile it, run it:

CODE

```
/* syscalls.c - open, write, read and close, using only system calls.
 * Build:  gcc -Wall -Wextra -o syscalls syscalls.c
 * Run:    ./syscalls
 * See the calls: strace ./syscalls
 */
#include <fcntl.h>      /* open */
#include <unistd.h>     /* write, read, close */
#include <string.h>     /* strlen */
#include <stdio.h>      /* printf */
```

```c
int main(void) {
    const char *msg = "hello, kernel\n";

    /* open() is a system call: ask the kernel for a file, get back a descriptor */
    int fd = open("note.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644);
    if (fd < 0) { perror("open"); return 1; }
    printf("open returned descriptor %d\n", fd);

    /* write() is a system call: hand the bytes to the kernel */
    ssize_t n = write(fd, msg, strlen(msg));
    printf("write sent %zd bytes\n", n);
    close(fd);

    /* read it back, through the kernel again */
    fd = open("note.txt", O_RDONLY);
    char buf[64];
    n = read(fd, buf, sizeof(buf) - 1);
    buf[n] = '\0';
    printf("read got %zd bytes: %s", n, buf);
    close(fd);
    return 0;
}
```

TERMINAL

```
$ gcc -Wall -Wextra -o syscalls syscalls.c
$ ./syscalls
```

Expected output:

EXPECTED OUTPUT

```
open returned descriptor 3
write sent 14 bytes
read got 14 bytes: hello, kernel
```

Now watch the same program from outside, with strace. You will see your three C calls become real system calls:

TERMINAL

```
$ strace -e trace=openat,write,read,close ./syscalls
```

Among the setup lines you will find yours:

TERMINAL

```
openat(AT_FDCWD, "note.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644) = 3
write(3, "hello, kernel\n", 14)         = 14
close(3)                                = 0
openat(AT_FDCWD, "note.txt", O_RDONLY)  = 3
read(3, "hello, kernel\n", 63)          = 14
close(3)                                = 0
```

**Observe (write in report.md)**

The descriptor returned by the first openat is 3, not 0, 1 or 2. Why are those three already taken?